Software Testing Documentation for Aktour ViaBalkan

Project Team:

Furkan Duman, Dogukan Yurtturk, Eren Çapar, Can Aysen

Date: March 31, 2026

Contents

1 Introduction **2**

2 Technology Stack & Configuration **2**

3 Test Strategy **3**
3.1 Unit Tests (Service Layer) . . . 3
3.2 End-to-End Tests (HTTP + Security) . . . 3

4 Unit Test Catalogue **3**
4.1 CustomerServiceTest - 10 tests . . . 3
4.2 ExpenseServiceTest - 10 tests . . . 4
4.3 GuideServiceTest - 11 tests . . . 5
4.4 RouteServiceTest - 13 tests . . . 5
4.5 TollServiceTest - 7 tests . . . 6
4.6 TourServiceTest - 13 tests . . . 7
4.7 UserServiceTest - 6 tests . . . 8
4.8 VehicleServiceTest - 9 tests . . . 8
4.9 WaypointServiceTest - 7 tests . . . 9

5 End-to-End Test Catalogue **9**
5.1 Representative E2E Code . . . 10

6 How to Run **10**

7 Summary **11**

1. Introduction

This document provides a complete, test-by-test description of the automated test suite developed for the **Aktour ViaBalkan Management System**. The suite lives on the dedicated `tests` branch of the repository and targets the `tur-backend` module - a Spring Boot REST API that manages tours, guides, vehicles, customers, expenses, routes, waypoints, tolls, and user accounts.

The suite is organised into two layers:

- **Unit tests** (86 cases) validate the business logic of the service layer in isolation. Repositories, mappers, and the password encoder are replaced with Mockito mocks.
- **End-to-end tests** (15 cases) drive the full HTTP stack through `MockMvc` - exercising JSON (de)serialization, Spring Security, JWT authentication, and role-based authorization against an in-memory H2 database.

In total the suite contains **101 automated tests**. Sections 4 and 5 enumerate every individual test case together with the scenario it covers and its expected outcome.

2. Technology Stack & Configuration

Component	Version / Notes
Language	Java 21
Framework	Spring Boot 3.2.4
Build tool	Apache Maven
Test framework	JUnit 5 (Jupiter)
Mocking library	Mockito (`@Mock`, `@InjectMocks`, `MockitoExtension`)
Assertions	AssertJ (`assertThat`, `assertThatThrownBy`)
Integration testing	`spring-boot-starter-test` + `MockMvc`
Test database	H2 in-memory (MySQL compatibility mode)
Security	Spring Security + JWT (jjwt 0.12.5)

A dedicated Spring profile `test` (`src/test/resources/application-test.properties`) configures the end-to-end suite. It swaps the production MariaDB/MySQL datasource for an in-memory H2 database with `ddl-auto=create-drop` so every run starts from a clean schema, defines a fixed Base64 JWT secret with a 1-hour expiry, and seeds two administrator accounts (`admin1`, `admin2`) through the application's `DataInitializer`. No external services are required to run either layer of the suite.

3. Test Strategy

3.1. Unit Tests (Service Layer)

Every service class is tested in isolation following the **Arrange - Act - Assert** pattern:

1. **Arrange** - collaborators are stubbed with `when(...).thenReturn(...)`; sample entities and DTOs are prepared in a `@BeforeEach` setup method.
2. **Act** - the service method under test is invoked.
3. **Assert** - the returned DTO is checked with AssertJ, and side effects (`save`, `delete`) are confirmed with `verify(...)`; for negative paths the thrown exception type and message are asserted, and unwanted persistence is excluded with `verify(repo, never())...`.

Each service is covered by both **Positive** (happy-path) and **Negative** (error / not-found) scenarios.

3.2. End-to-End Tests (HTTP + Security)

`AuthFlowE2ETest` is annotated with `@SpringBootTest`, `@AutoConfigureMockMvc`, and `@ActiveProfiles("test")`. It issues real JSON HTTP requests via `MockMvc` and asserts on HTTP status codes and response fields (`jsonPath`). Guide accounts are created on the fly through the admin API, using an `AtomicInteger` sequence to keep test runs independent and repeatable.

4. Unit Test Catalogue

The tables below enumerate **every** unit test method, grouped by service class. The *Type* column marks each case as **Positive** or **Negative**.

4.1. CustomerServiceTest - 10 tests

`org.example.service.CustomerServiceTest`

#	Test method	Type	Verifies
1	`getCustomersByTour_returnsFilteredList`	Positive	Returns the customers belonging to a given tour, mapped to DTOs.
2	`getCustomersByTour_noCustomers_returnsEmptyList`	Negative	Returns an empty list when the tour has no customers.
3	`getCustomerById_found_returnsDTO`	Positive	Returns the customer DTO for an existing id.
4	`getCustomerById_notFound_throwsRuntimeException`	Negative	Throws `RuntimeException` (message contains the id) when not found.
5	`createCustomer_tourExists_savesAndReturnsDTO`	Positive	Persists a new customer under an existing tour and returns its DTO.

#	Test method	Type	Verifies
6	`createCustomer_tourNotFound_throwsRuntimeException`	Negative	Throws when the parent tour does not exist.
7	`updateCustomer_found_updatesAndReturns`	Positive	Updates an existing customer via the mapper and returns the DTO.
8	`updateCustomer_notFound_throwsRuntimeException`	Negative	Throws when updating a non-existing customer.
9	`deleteCustomer_exists_deletesSuccessfully`	Positive	Deletes an existing customer by id.
10	`deleteCustomer_notFound_throwsRuntimeException`	Negative	Throws when deleting a non-existing customer.

4.2. ExpenseServiceTest - 10 tests

`org.example.service.ExpenseServiceTest`

#	Test method	Type	Verifies
1	`getExpensesByTour_returnsFilteredList`	Positive	Returns the expenses of a given tour mapped to DTOs.
2	`getExpensesByTour_noExpenses_returnsEmptyList`	Negative	Returns an empty list when the tour has no expenses.
3	`getExpenseById_found_returnsDTO`	Positive	Returns the expense DTO for an existing id.
4	`getExpenseById_notFound_throwsRuntimeException`	Negative	Throws when the expense id is unknown.
5	`createExpense_tourExists_savesAndReturnsDTO`	Positive	Persists a new expense under an existing tour.
6	`createExpense_tourNotFound_throwsRuntimeException`	Negative	Throws when the parent tour does not exist.
7	`updateExpense_found_updatesAndReturns`	Positive	Updates an existing expense and returns the DTO.
8	`updateExpense_notFound_throwsRuntimeException`	Negative	Throws when updating a non-existing expense.
9	`deleteExpense_exists_deletesSuccessfully`	Positive	Deletes an existing expense by id.
10	`deleteExpense_notFound_throwsRuntimeException`	Negative	Throws when deleting a non-existing expense.

4.3. GuideServiceTest - 11 tests

`org.example.service.GuideServiceTest` (uses a mocked Spring Security context)

#	Test method	Type	Verifies
1	`getAllGuides_returnsMappedList`	Positive	Returns all guides mapped to response DTOs.
2	`getAllGuidesSummary_returnsSummaryList`	Positive	Returns all guides mapped to summary DTOs.
3	`getGuideById_found_returnsDTO`	Positive	Returns a guide DTO for an existing id.
4	`getGuideById_notFound_throwsResourceNotFoundException`	Negative	Throws `ResourceNotFoundException` for an unknown id.
5	`getMyProfile_existingUser_returnsDTO`	Positive	Resolves the authenticated user and returns their guide profile.
6	`getMyProfile_notFound_throwsResourceNotFoundException`	Negative	Throws when the authenticated user has no guide profile.
7	`createGuide_newUsername_createsUserAndGuide`	Positive	Creates the backing user (encoded password) and the guide entity.
8	`createGuide_duplicateUsername_throwsIllegalArgumentException`	Negative	Throws `IllegalArgumentException` when the username already exists.
9	`updateMyProfile_existingGuide_updatesAndReturns`	Positive	Updates the authenticated guide's own profile.
10	`deleteGuide_found_deletesGuideAndUser`	Positive	Deletes both the guide and its linked user account.
11	`deleteGuide_notFound_throwsResourceNotFoundException`	Negative	Throws when deleting a non-existing guide.

4.4. RouteServiceTest - 13 tests

`org.example.service.RouteServiceTest`

#	Test method	Type	Verifies
1	`getAllRoutes_returnsMappedSummaryList`	Positive	Returns all routes as summary DTOs.
2	`getAllRoutes_emptyRepository_returnsEmptyList`	Negative	Returns an empty list when no routes exist.

#	Test method	Type	Verifies
3	`getRouteById_found_returnsDTO`	Positive	Returns a route DTO for an existing id.
4	`getRouteById_notFound_throwsResourceNotFoundException`	Negative	Throws for an unknown route id.
5	`createRoute_noRelations_savesAndReturnsDTO`	Positive	Creates a route with no waypoint/toll relations.
6	`createRoute_withWaypointIds_resolvesWaypoints`	Positive	Resolves and attaches default waypoints by id on creation.
7	`updateRoute_found_updatesAndReturns`	Positive	Updates an existing route and saves it.
8	`updateRoute_notFound_throwsResourceNotFoundException`	Negative	Throws when updating a non-existing route.
9	`deleteRoute_found_deletesSuccessfully`	Positive	Deletes an existing route.
10	`deleteRoute_notFound_throwsResourceNotFoundException`	Negative	Throws when deleting a non-existing route.
11	`addWaypointToRoute_notAlreadyPresent_addsAndSaves`	Positive	Adds a new waypoint to a route and persists it.
12	`addWaypointToRoute_alreadyPresent_doesNotDuplicate`	Negative	Does not duplicate a waypoint already on the route.
13	`removeWaypointFromRoute_present_removesAndSaves`	Positive	Removes a waypoint from a route and persists it.

4.5. TollServiceTest - 7 tests

`org.example.service.TollServiceTest`

#	Test method	Type	Verifies
1	`getAllTolls_returnsList`	Positive	Returns all tolls mapped to DTOs.
2	`getTollById_existing_returnsDto`	Positive	Returns a toll DTO for an existing id.
3	`getTollById_nonExisting_throws`	Negative	Throws `ResourceNotFoundException` ("Toll not found").
4	`createToll_savesAndReturnsDto`	Positive	Persists a new toll and returns its DTO.
5	`updateToll_nonExisting_throws`	Negative	Throws when updating a non-existing toll.

#	Test method	Type	Verifies
6	`deleteToll_nonExisting_throws`	Negative	Throws and never calls `deleteById` for an unknown toll.
7	`deleteToll_existing_deletes`	Positive	Deletes an existing toll by id.

4.6. TourServiceTest - 13 tests

`org.example.service.TourServiceTest`

#	Test method	Type	Verifies
1	`getAllTours_returnsMappedList`	Positive	Returns all tours as summary DTOs.
2	`getAllTours_emptyRepository_returnsEmptyList`	Negative	Returns an empty list when no tours exist.
3	`getTourById_found_returnsDTO`	Positive	Returns a tour DTO for an existing id.
4	`getTourById_notFound_throwsResourceNotFoundException`	Negative	Throws for an unknown tour id.
5	`createTour_withGuideOnly_savesAndReturnsDTO`	Positive	Creates a tour resolving only the guide relation.
6	`createTour_withVehicle_resolvesVehicleRelation`	Positive	Creates a tour resolving guide and vehicle relations.
7	`updateTour_found_updatesAndReturns`	Positive	Updates an existing tour and saves it.
8	`updateTour_notFound_throwsResourceNotFoundException`	Negative	Throws when updating a non-existing tour.
9	`deleteTour_exists_deletesSuccessfully`	Positive	Deletes an existing tour.
10	`deleteTour_notFound_throwsResourceNotFoundException`	Negative	Throws when deleting a non-existing tour.
11	`addWaypoint_tourAndWaypointExist_addsWaypointAndSaves`	Positive	Adds an extra waypoint to a tour and persists it.
12	`addWaypoint_waypointAlreadyPresent_doesNotDuplicate`	Negative	Does not duplicate a waypoint already on the tour.
13	`removeWaypoint_tourExists_removesWaypointAndSaves`	Positive	Removes an extra waypoint from a tour and persists it.

4.7. UserServiceTest - 6 tests

`org.example.service.UserServiceTest`

#	Test method	Type	Verifies
1	`getAllUsers_positive_returnsUserResponseList`	Positive	Returns all users mapped to response DTOs.
2	`getAllUsers_negative_noUsers_returnsEmptyList`	Negative	Returns an empty list when there are no users.
3	`getUserById_positive_existingId_returnsUserResponse`	Positive	Returns a user DTO for an existing id.
4	`getUserById_negative_nonExistingId_throwsResourceNotFoundException`	Negative	Throws `ResourceNotFoundException` for an unknown id.
5	`deleteUser_positive_existingId_deletesSuccessfully`	Positive	Deletes an existing user by id.
6	`deleteUser_negative_nonExistingId_throwsResourceNotFoundException`	Negative	Throws and never calls `deleteById` for an unknown user.

4.8. VehicleServiceTest - 9 tests

`org.example.service.VehicleServiceTest`

#	Test method	Type	Verifies
1	`getAllVehicles_returnsMappedSummaryList`	Positive	Returns all vehicles as summary DTOs.
2	`getAllVehicles_emptyRepository_returnsEmptyList`	Negative	Returns an empty list when no vehicles exist.
3	`getVehicleById_found_returnsDTO`	Positive	Returns a vehicle DTO for an existing id.
4	`getVehicleById_notFound_throwsResourceNotFoundException`	Negative	Throws for an unknown vehicle id.
5	`createVehicle_savesAndReturnsDTO`	Positive	Persists a new vehicle and returns its DTO.
6	`updateVehicle_found_updatesAndReturns`	Positive	Updates an existing vehicle and saves it.
7	`updateVehicle_notFound_throwsResourceNotFoundException`	Negative	Throws when updating a non-existing vehicle.
8	`deleteVehicle_exists_deletesSuccessfully`	Positive	Deletes an existing vehicle.
9	`deleteVehicle_notFound_throwsResourceNotFoundException`	Negative	Throws when deleting a non-existing vehicle.

4.9. WaypointServiceTest - 7 tests

`org.example.service.WaypointServiceTest`

#	Test method	Type	Verifies
1	`getAllWaypoints_returnsList`	Positive	Returns all waypoints mapped to DTOs.
2	`getWaypointById_existing_returnsDto`	Positive	Returns a waypoint DTO for an existing id.
3	`getWaypointById_nonExisting_throws`	Negative	Throws `ResourceNotFoundException` ("Waypoint not found").
4	`createWaypoint_savesAndReturnsDto`	Positive	Persists a new waypoint and returns its DTO.
5	`updateWaypoint_nonExisting_throws`	Negative	Throws when updating a non-existing waypoint.
6	`deleteWaypoint_nonExisting_throws`	Negative	Throws and never calls `deleteById` for an unknown waypoint.
7	`deleteWaypoint_existing_deletes`	Positive	Deletes an existing waypoint by id.

5. End-to-End Test Catalogue

`org.example.e2e.AuthFlowE2ETest` - 15 scenarios exercising the full HTTP, security, and validation pipeline.

ID	Scenario	Type	Expected
E2E-01	Login with valid admin credentials returns a JWT, username, and `role=ADMIN`.	Positive	200 OK
E2E-02	Login with a wrong password is rejected.	Negative	4xx
E2E-03	Login with an unknown username is rejected.	Negative	4xx
E2E-04	Protected endpoint (`GET /api/guides`) accessed with no token.	Negative	4xx
E2E-05	Protected endpoint accessed with a malformed / garbage token.	Negative	4xx
E2E-06	`GET /api/guides` with a valid admin token.	Positive	200 OK
E2E-07	Admin can view the user list (`GET /api/users`).	Positive	200 OK
E2E-08	Admin creates a new guide (`POST /api/guides`).	Positive	200 OK
E2E-09	A newly created guide can log in and receives `role=GUIDE`.	Positive	200 OK

ID	Scenario	Type	Expected
E2E-10	A guide token is denied access to user management.	Negative	403
E2E-11	A guide token is denied vehicle creation (`POST /api/vehicles`).	Negative	403
E2E-12	A guide can view the vehicle list (`GET /api/vehicles`).	Positive	200 OK
E2E-13	Request for a non-existing tour (`GET /api/tours/999999`).	Negative	404
E2E-14	Create a customer with a blank `fullName` (validation).	Negative	400
E2E-15	A guide can view their own profile (`GET /api/guides/me`).	Positive	200 OK

5.1. Representative E2E Code

```
@Test
@DisplayName("E2E-10 GUIDE cannot access user management -> 403")
void e2e10_getUsers_withGuideToken_isForbidden() throws Exception {
    mockMvc.perform(get("/api/users")
                    .header("Authorization", "Bearer " + guideToken()))
            .andExpect(status().isForbidden());
}
```

6. How to Run

From the `tur-backend` module directory:

```
# Run the full suite (unit + end-to-end)
mvn test

# Run a single class
mvn -Dtest=AuthFlowE2ETest test

# Run a single test method
mvn -Dtest=CustomerServiceTest#createCustomer_tourNotFound_throwsRuntimeException test
```

No external database is needed: unit tests run on Mockito mocks without a Spring context, and E2E tests run against the in-memory H2 instance from the `test` profile.

7. Summary

Test class	Layer	Tests
CustomerServiceTest	Unit	10
ExpenseServiceTest	Unit	10
GuideServiceTest	Unit	11
RouteServiceTest	Unit	13
TollServiceTest	Unit	7
TourServiceTest	Unit	13
UserServiceTest	Unit	6
VehicleServiceTest	Unit	9
WaypointServiceTest	Unit	7
Unit subtotal		**86**
AuthFlowE2ETest	End-to-End	15
Total automated tests		**101**

The combined suites give layered coverage of the Aktour ViaBalkan backend. The 86 unit tests guarantee the correctness of the service-layer business logic - create, read, update, delete, and relationship management together with their error paths - while the 15 end-to-end tests verify that the assembled application correctly enforces authentication, JWT validation, role-based authorization, input validation, and resource-not-found handling across the REST API.